



A.D. 1308
unipg

DIPARTIMENTO
DI MATEMATICA E INFORMATICA

CORSO DI LAUREA (TRIENNALE) IN INFORMATICA

Tesi di Laurea

Sviluppo di un framework per l'ottimizzazione
dell'output OCR tramite Large Language Models:
un'analisi sperimentale sull'efficacia del Prompt
Engineering.

Laureando

Samuele Costantini

Relatore/i

Prof. Leonardo Mostarda

Anno Accademico 2024/2025

1 Presentazione del progetto.....	4
1.1 Obiettivo generale della tesi	
L'obiettivo primario del presente lavoro di tesi è l'analisi e la validazione sperimentale delle metodologie più efficaci per l'ottimizzazione dell'accuratezza nelle pipeline di estrazione dati ibride (OCR integrato con Large Language Models).....	4
1.2 Inquadramento del problema e tecnologie di riferimento.....	4
1.3 Contesto e strumenti utilizzati.....	6
2 Stack Tecnologico e Scelte Implementative.....	8
2.1 Descrizione.....	8
2.2 Laravel.....	9
2.3 Interfaccia Utente e Visualizzazione Dati: L'ecosistema Filament.....	9
2.4 Modellazione dei Dati e Schema Relazionale.....	10
2.4.1 Definizione della Struttura (Configuration Layer).....	10
2.4.2 Core Execution Engine Il cuore pulsante del sistema è l'entità Run.....	10
2.4.3 Evaluation & Benchmarking.....	11
3 Progettazione logica e strutturale del sistema.....	12
3.1 Architettura Software e Adozione dei Design Pattern.....	12
3.2 Descrizione del Data Model.....	14
3.3 Entità di Configurazione e Semantica.....	14
3.4 Entità Core (Esecuzione).....	14
3.5 Entità di Valutazione (Benchmarking).....	15
4 Commands, Actions e Services.....	16
4.1 Le Actions.....	16
4.2 I Services (Microservizi Applicativi).....	18
4.3 Gestione Asincrona (Jobs).....	18
5. Selezione dello Stack Tecnologico AI.....	19
5.1 Il Componente OCR: Amazon Textract.....	19
5.2 L'integrazione del Large Language Model (LLM).....	21
6 Descrizione pipeline di estrazione ed elaborazione.....	22
7 Prompt Engineering.....	27
7.1 Tecniche di prompt Engineering.....	27
7.1.1 Zero-shot prompting.....	27
7.1.2 Few-Shot Prompting.....	27
7.1.3 Chain-of-Thought (CoT) Prompting.....	28
7.1.4 Forensic + CoT + Structural Awareness (Hybrid).....	28
7.1.5 Emotion Prompting.....	28

8 Evaluation.....	30
8.1 Dataset.....	30
8.2 Benchmark.....	31
8.3 Definizione della Metrica di Valutazione (Score).....	33
8.3.1 Calcolo dello score.....	33
8.3.2 Considerazioni sugli algoritmi scelti.....	37
9 Risultati.....	38
9.1 Presentazione dei risultati.....	39
9.2 Analisi dei risultati.....	40
9.2.1 Analisi sullo score medio dei prompt.....	40
9.2.2 Sensibilità della Metrica.....	46
9.3 Analisi specifiche sui risultati di ogni prompt.....	47
9.3.1 Analisi di Zero-shot.....	47
9.3.2 Analisi di Few-shot.....	47
9.3.3 Analisi del Chain-of-Thought.....	48
9.3.4 Analisi del Forensic + CoT + Structural Awareness (Hybrid).....	49
9.3.5 Analisi dello Zero-Shot with Emotional stimuli.....	50
10 Conclusione.....	51
Bibliografia e Sitografia.....	53

Introduzione

Il presente lavoro di tesi trae origine dall'analisi di specifiche criticità riscontrate in ambito operativo aziendale durante il mio tirocinio, relative ai processi di validazione automatizzata dei dati.

L'obiettivo primario è stato identificato nell'automazione del confronto tra i dati strutturati (input utente) e i dati non strutturati presenti nella documentazione fiscale (immagini di documenti di acquisto). L'assenza sul mercato di soluzioni in grado di garantire le performance richieste ha reso necessaria la progettazione di un'architettura OCR (Optical Character Recognition) customizzata.

Durante le fasi preliminari di sviluppo e ricerca, è emerso come l'integrazione di tecniche avanzate di Prompt Engineering e di pre-processing dell'immagine incidesse significativamente sull'accuratezza dell'estrazione. Tali evidenze empiriche, unite alla vastità dello stato dell'arte in materia, hanno motivato la scelta di isolare il problema in un ambiente controllato. Questo elaborato si propone dunque di sistematizzare l'approccio sperimentale, realizzando un framework di benchmark dedicato per testare e validare diverse strategie di ottimizzazione dell'output OCR tramite LLM.

1 Presentazione del progetto

1.1 Obiettivo generale della tesi

L'obiettivo primario del presente lavoro di tesi è l'analisi e la validazione sperimentale delle metodologie più efficaci per l'ottimizzazione dell'accuratezza nelle pipeline di estrazione dati ibride (OCR integrato con Large Language Models).

Nello specifico, la ricerca si focalizza sull'impatto del Prompt Engineering come strumento di correzione semantica e strutturale degli output grezzi forniti dai sistemi di riconoscimento ottico. Lo studio prevede la conduzione di un'analisi comparativa rigorosa: diverse strategie di prompting verranno valutate all'interno di un framework di testing standardizzato, garantendo la riproducibilità dei risultati e l'oggettività delle metriche di valutazione adottate.

1.2 Inquadramento del problema e tecnologie di riferimento

La conversione di documenti analogici in dati digitali strutturati rappresenta una delle sfide classiche dell'informatica applicata. Al centro di questo processo si colloca la tecnologia di Optical Character Recognition (OCR), definita in letteratura da Mittal e Garg come la conversione informatizzata di testo proveniente da fonti eterogenee (documenti manoscritti, stampati o immagini naturali) in formati digitali editabili. [1]

L'OCR si inserisce nel più ampio dominio del Digital Image Processing (DIP), una branca dell'informatica che utilizza algoritmi complessi per l'elaborazione di segnali visivi. Sebbene le applicazioni del DIP spaziano dalla diagnostica medica per immagini (es. scansioni PET) alla robotica industriale, nel contesto dell'automazione aziendale l'obiettivo primario rimane il pattern recognition: la capacità della macchina di identificare caratteri e strutture all'interno di un documento.

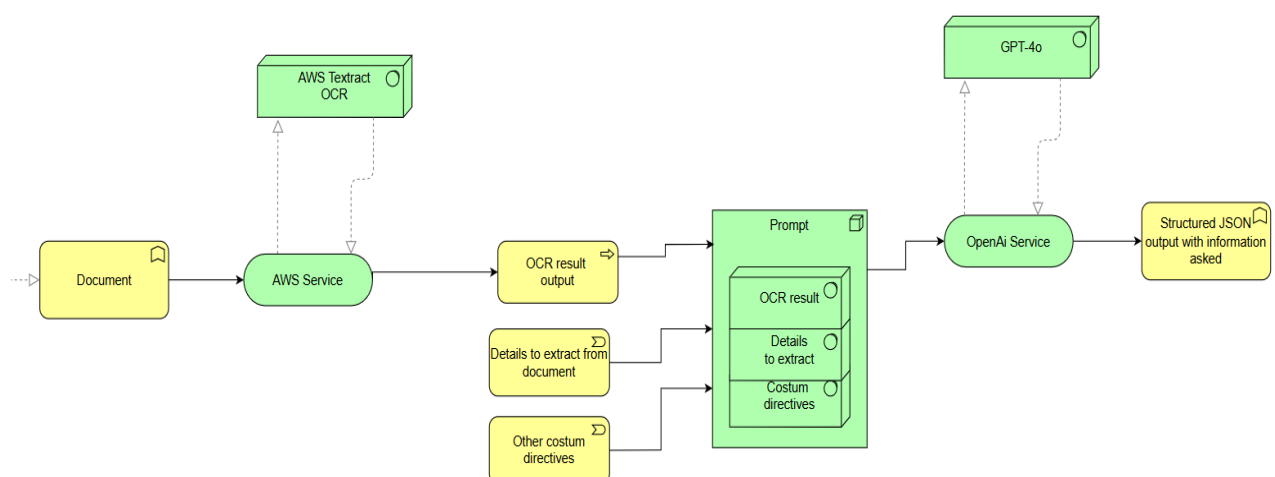
Tuttavia, l'approccio OCR tradizionale presenta limitazioni intrinseche quando si tratta di interpretare la semantica del dato o di gestire layout non standardizzati. Per affrontare tale

complessità, il presente progetto adotta un'architettura ibrida che integra due tecnologie allo stato dell'arte:

1. Amazon AWS Textract (OCR Engine): Selezionato per le sue capacità di Machine Learning avanzato, Textract supera il concetto di semplice riconoscimento dei caratteri, offrendo funzionalità native per l'estrazione di elementi di layout (tabelle, form) e relazioni spaziali all'interno del documento.

2. GPT-4o (Semantic Processing): Per la fase di post-elaborazione e strutturazione del dato, si fa ricorso ai Large Language Models (LLM). Nello specifico, l'utilizzo delle API di OpenAI permette di sfruttare un modello addestrato su vasti corpora testuali tramite apprendimento auto-supervisionato. Il ruolo dell'LLM in questa architettura è quello di agire come "motore di ragionamento", correggendo gli errori di lettura dell'OCR e normalizzando l'output in formati strutturati (JSON) pronti per l'integrazione in database.

Il sistema utilizzato in questo progetto segue il seguente schema:



1.3 Contesto e strumenti utilizzati

La piattaforma di test è stata sviluppata utilizzando il framework PHP Laravel, strumento per lo sviluppo di applicativi web sia per la parte back-end che per la parte front-end.

Altri strumenti e librerie interessanti usate sono: Livewire con Blade (per il front-end), Filament (per la dashboard), MySQL (per il Database).

Elenco di librerie e dipendenze utilizzate:

Libreria	Versione	Funzionalità principale
php	8.2	Linguaggio di programmazione utilizzato per l'intera applicazione
aws/aws-sdk-php	3.339	SDK per l'interazione con i servizi AWS
filament/filament	3.2	Pannello di amministrazione per Laravel
filament/notifications	3.2	Sistema di notifiche integrato in Filament
filament/spatie-laravel-media-library-plugin	3.3	Estensione di Filament per la gestione di media con Spatie Media Library
fpdf/fpdf	1.86	Generazione di documenti PDF
justinrainbow/json-schema	6.2	Validazione di dati JSON secondo uno schema definito
laravel/framework	11.31	Framework PHP principale per lo sviluppo dell'applicazione
laravel/tinker	2.9	REPL per l'esecuzione interattiva di codice Laravel
livewire/livewire	3.4	Creazione di componenti interattivi lato server in Laravel
livewire/volt	1.6	Template engine per Livewire
maatwebsite/excel	3.1	Lettura, scrittura e gestione di file Excel
openai-php/laravel	0.10.2	Integrazione dell'API OpenAI con Laravel
php-http/discovery	1.20	Individuazione automatica dei client HTTP compatibili

prism-php/prism	0.67.0	Parsing e gestione di documenti complessi
saloonphp/laravel-plugin	3.0	Plugin per l'integrazione di Saloon con Laravel
saloonphp/saloon	3.0	HTTP client moderno e modulare per PHP
setasign/fpdf	2.6	Importazione e manipolazione di PDF esistenti
spatie/laravel-medialibrary	11.12	Gestione avanzata di file e media all'interno di Laravel
spatie/pdf-to-image	3.1	Conversione di file PDF in immagini
symfony/http-client	7.2	Client HTTP basato sul framework Symfony
ufree/sqlite3	1.0	Gestione di database SQLite in PHP

2 Stack Tecnologico e Scelte Implementative

2.1 Descrizione

Per lo sviluppo del back-end e dell'orchestrazione dei processi di benchmark, la scelta è ricaduta sull'ecosistema PHP 8.2 [2] [3] supportato dal framework Laravel 11. Tale decisione non è stata dettata da mere preferenze stilistiche, ma dalla necessità di gestire flussi di lavoro complessi e latenze di rete intrinseche alle chiamate API esterne.

A differenza di un approccio procedurale classico, l'architettura è stata progettata sfruttando le caratteristiche avanzate del linguaggio e del framework:

Gestione Asincrona tramite Queues: Le operazioni di OCR (AWS Textract) e di inferenza semantica (GPT-4o) sono processi time-consuming che, se eseguiti in modo sincrono, bloccherebbero il thread principale rendendo l'interfaccia utente non responsiva. L'utilizzo del sistema di Job Queues di Laravel ha permesso di disaccoppiare l'invio della richiesta dalla sua elaborazione. Il `BulkRunJob` (cfr capitolo 4.3 Gestione Asincrona (Jobs)) viene spacciato in una coda di lavorazione gestita da worker dedicati, garantendo scalabilità e resilienza anche a fronte di carichi di lavoro massivi.

Service Container e Dependency Injection: Per garantire la modularità e la testabilità del codice, i servizi di integrazione (`AwsTextractService` e `OpenAiService`) sono stati implementati sfruttando il pattern della Dependency Injection. Questo approccio permette di gestire le dipendenze in modo dichiarativo e facilita l'eventuale sostituzione dei provider (es. cambiare AWS con Google Document AI) senza dover rifattorizzare la logica di business principale.

Tipizzazione Forte e DTO: Sfruttando le feature di PHP 8.2, il sistema fa ampio uso di tipizzazione forte (return types, union types) per la gestione dei dati strutturati. Questo riduce drasticamente i bug a runtime, specialmente durante il parsing delle risposte JSON complesse provenienti dai modelli LLM.

2.2 Laravel

Laravel è un framework open-source per lo sviluppo di applicazioni web, caratterizzato da una sintassi chiara e leggibile che facilita la scrittura di codice manutenibile. Un framework web fornisce una struttura di base e una serie di strumenti riutilizzabili, consentendo agli sviluppatori di concentrarsi sulla logica applicativa piuttosto che sui dettagli infrastrutturali di basso livello.

Laravel include un insieme completo di funzionalità avanzate, tra cui un sistema di dependency injection, un livello di astrazione del database tramite Eloquent ORM, la gestione di code e processi pianificati, strumenti integrati per test unitari e di integrazione, e un sistema modulare per la gestione di pacchetti e dipendenze. Grazie a queste caratteristiche, Laravel si presta particolarmente allo sviluppo di applicazioni complesse e scalabili, fornendo al contempo un ambiente coerente e strutturato che riduce il rischio di errori e favorisce la qualità del software. [4]

2.3 Interfaccia Utente e Visualizzazione Dati: L'ecosistema Filament

Data la natura sperimentale del progetto, incentrata sull'analisi di grandi moli di dati (risultati dei benchmark, confronti tra prompt), la progettazione dell'interfaccia utente (UI) ha privilegiato l'usabilità e la rapidità di consultazione rispetto all'estetica puramente decorativa.

Per tali ragioni, l'architettura di frontend è stata realizzata mediante Filament PHP, un framework open-source basato sullo stack TALL (Tailwind, Alpine, Laravel, Livewire). Questa scelta strategica ha offerto due vantaggi tecnici determinanti:

Sviluppo Data-Centric: Filament fornisce astrazioni di alto livello per la generazione automatica di tabelle complesse e form di input. Questo ha permesso di implementare rapidamente le viste necessarie per il confronto dei risultati (Ground Truth vs Output LLM) e per la gestione del dataset, mantenendo il focus dello sviluppo sulla logica algoritmica di back-end piuttosto che sulla costruzione dei componenti UI.

2. Reattività Server-Side con Livewire: L'integrazione nativa con Livewire ha reso possibile la creazione di interfacce dinamiche senza la complessità di una Single Page Application (SPA) separata in JavaScript. In particolare, Livewire è stato sfruttato per gestire il

feedback in tempo reale durante i processi asincroni, come l'aggiornamento delle barre di avanzamento durante l'esecuzione dei Job di estrazione massiva, garantendo un'esperienza utente fluida durante le fasi di benchmark intensive. [5][6]

2.4 Modellazione dei Dati e Schema Relazionale

L'architettura del database è stata progettata per garantire la totale separazione tra la definizione del test e la sua esecuzione, utilizzando un approccio relazionale normalizzato. Analizzando lo schema fisico, si evidenziano tre macro-aree funzionali.

2.4.1 Definizione della Struttura (Configuration Layer)

Al vertice della gerarchia troviamo il DetailSet, che agisce come un catalogo di tipologie documentali (es. "Scontrino", "Fattura"). Esso raggruppa molteplici DocumentDetail, ovvero le singole entità informative da estrarre (es. "Data", "Importo"). Questa struttura permette di associare a ogni Document un set di regole predefinito tramite la foreign key detail_set_id.

2.4.2 Core Execution Engine Il cuore pulsante del sistema è l'entità Run.

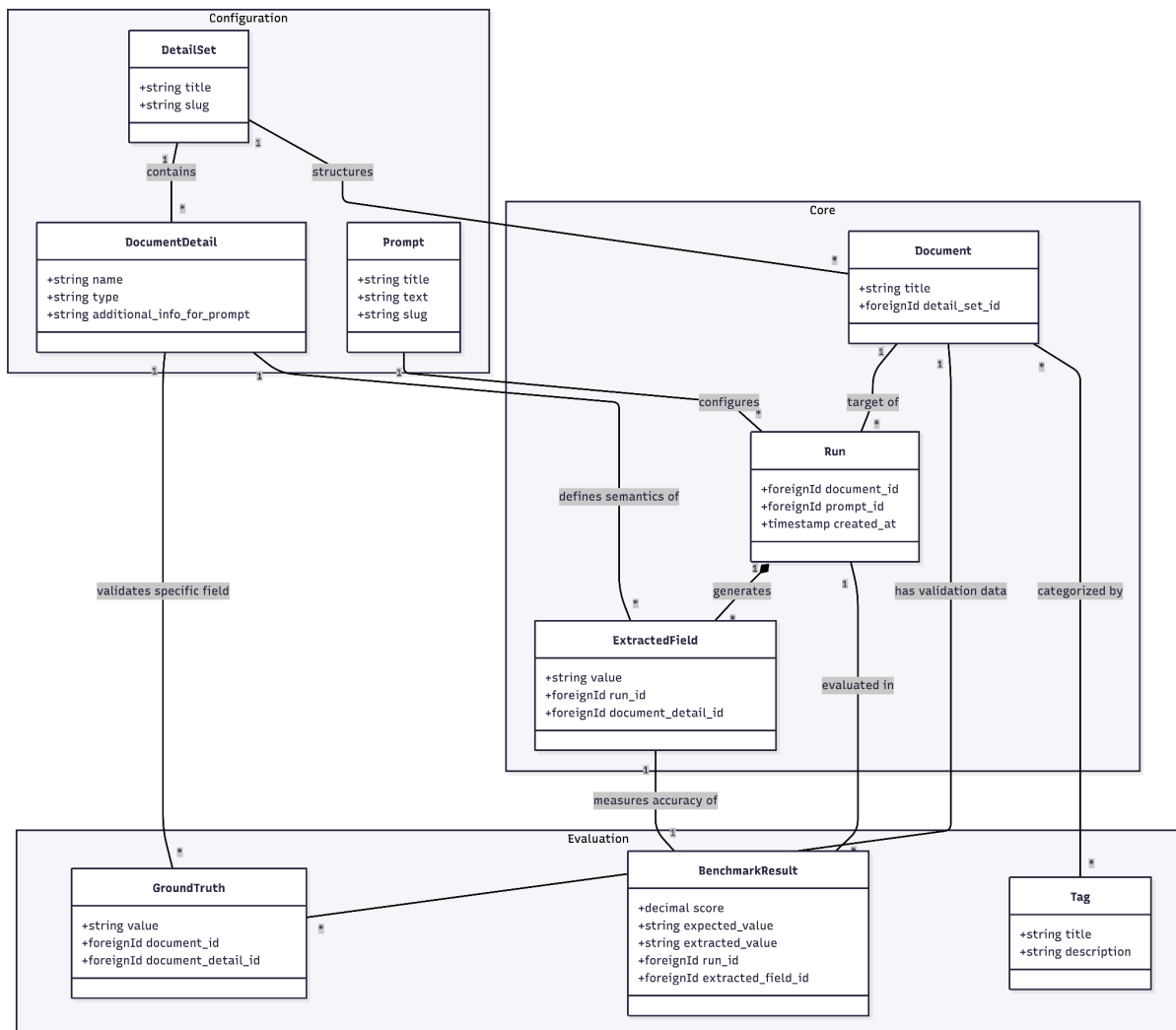
Questa tabella funge da pivot tra il Document (l'input) e il Prompt (la logica di estrazione), storicizzando ogni singolo esperimento. È fondamentale notare come la relazione tra documento e prompt non sia diretta, ma mediata dalla Run: questo permette di testare lo stesso documento con infiniti prompt diversi senza ridondanza dei dati. I risultati dell'inferenza vengono salvati nella tabella extracted_fields, che collega il valore estratto (value) alla specifica Run e al Dettaglio di riferimento.

2.4.3 Evaluation & Benchmarking.

Per la validazione scientifica, il sistema introduce due entità distinte

- GroundTruth: Contiene i valori "reali" validati manualmente, collegati direttamente al documento e al dettaglio specifico.
- BenchmarkResult: Questa tabella materializza l'esito del confronto. Oltre allo score numerico (0.0 - 1.0), storicizza anche il valore atteso ed estratto al momento del test, permettendo analisi a posteriori anche se il Ground Truth dovesse cambiare in futuro.

Infine, l'uso di una relazione many-to-many per i Tags consente una categorizzazione trasversale dei documenti (es. "Low Quality", "Handwritten"), essenziale per analizzare le performance dei prompt su cluster di dati specifici.



3 Progettazione logica e strutturale del sistema

3.1 Architettura Software e Adozione dei Design Pattern

Per la definizione dell'architettura software, si è optato per una rigorosa implementazione del Command Pattern [8], un approccio architetturale a cui il framework Laravel si presta con naturalezza.

Come noto in ingegneria del software, un Design Pattern rappresenta una soluzione progettuale riutilizzabile per risolvere problematiche ricorrenti nei sistemi orientati agli oggetti [7]. La scelta di questo specifico pattern comportamentale non è casuale, ma risponde alle esigenze intrinseche del progetto: realizzare una piattaforma di testing per la validazione di prompt AI. Tale sistema richiede infatti un'elevata modularità e la capacità di scalare agilmente per integrare dataset più vasti e complessi, nonché moderne tecniche di prompt engineering. L'adozione del Command Pattern garantisce quindi la manutenibilità del codice nel tempo e facilita la collaborazione tra diversi sviluppatori, permettendo modifiche granulari senza compromettere l'integrità del sistema.

Nello specifico, il pattern Command permette di incapsulare le richieste come oggetti, strutturando l'applicazione attraverso azioni primitive (Actions) gestibili ed eseguibili in modo isolato. Il principio cardine è il disaccoppiamento (decoupling): si separano nettamente gli oggetti che invocano le operazioni da quelli che sanno come eseguirle. L'architettura risultante prevede diverse classi di oggetti Command che, implementando una comune interfaccia, definiscono azioni distinte. Questa astrazione consente di passare i command object come parametri ad altri oggetti, i quali possono eseguirne la logica interna senza conoscerne i dettagli implementativi.

Dal punto di vista tecnologico, la scelta di Laravel è strategica. Il framework valorizza appieno il paradigma a oggetti di PHP, linguaggio che, a partire dalla versione 5, ha introdotto un object model completo, ispirandosi alla sintassi rigorosa di Java e C# e trasformandosi da semplice linguaggio di scripting a potente strumento Object-Oriented [9]. Questa evoluzione rende l'ambiente ideale per l'implementazione del pattern Command-Action descritto.

Struttura della codebase:

```
app/
├── Actions/
│   ├── RunBenchmarkAction.php
│   └── RunExtractionAction.php
├── Console/
│   ├── Commands/
│   │   └── AwsTexttractTest.php
├── Enums/
│   ├── AwsSignals.php
│   ├── DetailFormat.php
│   └── DetailType.php
├── Facades/
│   ├── DetailSchema.php
│   └── Evaluation.php
├── Filament/
│   └── ...
├── Http/
│   └── ...
├── Interfaces/
│   ├── AIService.php
│   └── OCRService.php
├── Jobs/
│   └── BulkRunJob.php
├── Livewire/
│   ├── Actions/
│   │   └── Logout.php
├── Models/
│   ├── BenchmarkResult.php
│   ├── DetailSet.php
│   ├── Document.php
│   ├── DocumentDetail.php
│   ├── ExtractedField.php
│   ├── ExtractionResult.php
│   ├── GroundTruth.php
│   ├── Prompt.php
│   ├── Run.php
│   ├── Tag.php
│   └── User.php
├── Providers/
│   ├── Filament/
│   │   ├── AdminPanelProvider.php
│   │   └── VoltServiceProvider.php
├── Services/
│   ├── AwsTexttractService.php
│   ├── DetailSchemaService.php
│   ├── EvaluationService.php
│   └── OpenAIService.php
└── AwsLowConfidenceError.php
```

3.2 Descrizione del Data Model

Il modello dati è stato progettato per garantire la massima flessibilità nella definizione dei test e una rigorosa tracciabilità dei risultati. Di seguito vengono descritte le principali entità del dominio, suddivise per responsabilità logica.

3.3 Entità di Configurazione e Semantica

Queste entità definiscono cosa il sistema deve cercare all'interno dei documenti, rendendo la piattaforma agnostica rispetto alla tipologia dei file analizzati.

Detail Set: Rappresenta l'astrazione della categoria documentale (es. "Scontrino", "Fattura", "Busta paga"). Agisce come un contenitore logico per un insieme di dettagli, definendo lo schema di estrazione che verrà applicato a tutti i documenti associati. Questa struttura permette di estendere il software a nuovi domini senza modificare il codice sorgente.

Document Detail: Definisce il singolo attributo atomico da estrarre (es. "Importo Totale", "Data Emissione"). Ogni dettaglio è vincolato a uno specifico Detail Set tramite relazione One-to-Many e contiene le istruzioni semantiche necessarie per guidare il prompt del Large Language Model nell'identificazione del dato.

Prompt: Costituisce la variabile indipendente dell'esperimento. Questa entità storicizza i diversi template di istruzioni (system prompt e user prompt) inviati a GPT-4o. La configurabilità dei prompt via software permette di testare diverse strategie di Prompt Engineering (es. Few-Shot, Chain-of-Thought) mantenendo invariato il resto della pipeline.

3.4 Entità Core (Esecuzione)

Queste entità gestiscono il flusso operativo e collegano i dati di input con le logiche di elaborazione.

Document: L'entità centrale del dataset. Rappresenta il file (immagine o PDF) sottoposto ad analisi. Oltre a gestire il riferimento al file fisico (tramite la libreria Spatie Media Library), il documento è associato a un Detail Set che ne determina la struttura informativa attesa.

Run: Rappresenta la singola istanza di esecuzione di un processo di estrazione. Ogni volta che il sistema elabora un Document utilizzando uno specifico Prompt, viene generata una Run. Questa entità funge da snapshot temporale, garantendo la tracciabilità dell'esperimento e permettendo di rieseguire test sullo stesso documento con prompt diversi senza sovrascrivere i dati storici.

Extracted Field: È l'entità che ospita l'output del sistema. Collega logicamente una Run (il contesto dell'estrazione) con il Document Detail (il dato cercato), memorizzando il valore grezzo restituito dal modello AI. Questa struttura granulare permette di analizzare le performance per singolo campo.

3.5 Entità di Valutazione (Benchmarking)

Il modulo finale del database, dedicato alla validazione dei risultati e alla generazione delle metriche.

Ground Truth: Rappresenta la "verità fondamentale", ovvero il valore reale e validato manualmente per ogni Document Detail di un dato documento. Costituisce il termine di paragone (Gold Standard) contro cui vengono confrontati i valori estratti dall'AI.

Benchmark Result: Questa entità materializza l'esito del confronto tra l'output del sistema (Extracted Field) e il dato reale (Ground Truth). Per ogni campo estratto, il sistema calcola e memorizza un punteggio di similarità (score) e registra sia il valore atteso che quello estratto, facilitando l'analisi degli errori.

Tag: Un sistema di etichettatura trasversale (relazione Many-to-Many) utilizzato per categorizzare i documenti (es. "Scansione sfocata", "Manoscritto", "Bassa risoluzione"). L'uso dei tag è fondamentale ai fini statistici per identificare correlazioni tra la qualità dell'input e il degrado delle performance del modello.

4 Commands, Actions e Services

In linea con l'architettura *Action-Oriented* adottata, la logica applicativa è stata suddivisa in componenti distinti per responsabilità: le Actions, che orchestrano i processi di business, e i Services, che incapsulano le integrazioni esterne e le logiche di supporto.

4.1 Le Actions

Le classi Action rappresentano le operazioni primitive del sistema.

RunBenchmarkAction: È l'azione deputata alla validazione massiva dei risultati. Il suo compito è calcolare lo "score" di accuratezza per ogni campo estratto (ExtractedField), confrontando il valore generato dall'AI con il valore reale (Ground Truth) associato al documento.

RunExtractionAction: Questa azione gestisce l'intera pipeline di estrazione del testo dalle immagini. Coordina la chiamata sequenziale ai servizi di OCR e di analisi semantica (LLM).

Di seguito viene mostrata l'implementazione del metodo handle della classe RunExtractionAction, che illustra il flusso operativo:

```
class RunExtractionAction
{
    public function handle(Document $document, Prompt $prompt): void
    {
        // Creazione dello snapshot dell'esecuzione (Run)
        $run = Run::create([
            'document_id' => $document->id,
            'prompt_id' => $prompt->id,
        ]);

        // 1. Estrazione del testo grezzo tramite servizio OCR (Textract)
        $extracted_text = app(AwsTextExtractService::class)
            ->textExtractor($document->getFirstMediaPath('document'));

        // 2. Analisi e strutturazione del dato tramite LLM (OpenAI GPT-4o)
        $structured_result = app(OpenAIService::class)
            ->readInvoice($extracted_text, $document, $prompt);

        // 3. Persistenza dei risultati nel database
        foreach ($structured_result as $key => $result) {
            // Recupero del dettaglio corrispondente alla chiave del JSON
            $document_detail = DocumentDetail::whereName($key)->first();

            if ($document_detail) {
                $document->extractedFields()->create([
                    'document_detail_id' => $document_detail->id,
                    'run_id' => $run->id,
                    'value' => $result,
                ]);
            }
        }
    }
}
```

php

4.2 I Services (Microservizi Applicativi)

I servizi gestiscono la complessità tecnica delle comunicazioni esterne e la preparazione dei dati.

AwsTextextractService: Gestisce l'interazione via API con il servizio AWS Textract. Oltre all'invio dell'immagine, include le funzioni di parsing della risposta JSON dell'OCR, ripulendo l'output e preparandolo per essere processato dal Large Language Model.

DetailSchemaService: Ha il compito di generare dinamicamente lo schema di validazione (Detail Schema). Partendo dai dettagli configurati per una specifica categoria di documenti (Detail Set), costruisce un oggetto JSON che istruisce l'LLM sulla struttura esatta dei dati da estrarre.

OpenAiService: Gestisce la comunicazione via API con il modello GPT-4o. Si occupa di assemblare il prompt finale (unendo le istruzioni di sistema, il testo OCR e lo schema dei dettagli) e di gestire la ricezione della risposta strutturata.

EvaluationService: Fornisce la libreria di algoritmi matematici necessari per il calcolo dello score, valutando il grado di similarità tra la risposta dell'LLM e il dato atteso (accuracy).

4.3 Gestione Asincrona (Jobs)

4.3.1 BulkRunJob: È un "Job", ovvero un'unità di lavoro serializzata che viene eseguita in background parallelamente al flusso principale dell'applicazione. Sfruttando il sistema di code (Queue) di Laravel, questi processi vengono eseguiti dal server secondo un paradigma FIFO (First-In-First-Out), garantendo che le esecuzioni massive non sovraccarichino le risorse o blocchino l'interfaccia utente. Le esecuzioni in una singola queue agiscono in serie, processando un documento alla volta [13]. In questo progetto, il Job viene utilizzato per gestire le chiamate API massive durante le fasi di test su ampi dataset.

5. Selezione dello Stack Tecnologico AI

5.1 Il Componente OCR: Amazon Textract

Per il modulo di Optical Character Recognition, la scelta architetturale è ricaduta sul servizio AWS Textract. Tale decisione è il risultato di un'analisi comparativa che ha soppesato l'accuratezza pura contro l'integrabilità e la robustezza dell'ecosistema.

Analizzando lo stato dell'arte, e in particolare lo studio di benchmarking condotto da Hegghammer T., si evidenzia come il panorama degli OCR sia dominato da tre competitor principali: Google Document AI, Amazon Textract e Tesseract (soluzione open source). I dati empirici riportati nello studio mostrano che:

Google Document AI registra il tasso di errore (Error Rate) più basso, posizionandosi come leader in termini di accuratezza pura sui caratteri latini.

Amazon Textract si colloca in una posizione mediana, con performance superiori a Tesseract e una notevole robustezza nella gestione di testi multilingua (superando Google in specifici test su set di caratteri non latini, come l'Arabo) [10].

Tesseract, pur essendo gratuito e locale, mostra i tassi di errore più elevati, rendendolo meno idoneo per applicazioni critiche senza un pesante pre-processing delle immagini. [10]

Sebbene Textract non detenga il primato assoluto nell'accuratezza del singolo carattere, la sua adozione in questo progetto è giustificata da un compromesso ingegneristico: la qualità della suite SDK di Amazon e la stabilità delle API permettono un'integrazione fluida nel backend Laravel. Questo garantisce una manutenibilità superiore e riduce i tempi di sviluppo, offrendo un rapporto costi/benefici ottimale per la natura prototipale del sistema, senza compromettere in modo significativo la leggibilità del documento.

Data: Single-column text in historical book scans with noise added artificially (n=42,504; 322 per engine and noise type).
 Noise codes: 'col'=colour, 'bin'=binary, 'blur'=blur, 'weak'=weak ink, 'snp'=salt&pepper, 'wm'=watermark, 'scrib'=scribbles, 'ink'=ink stains.

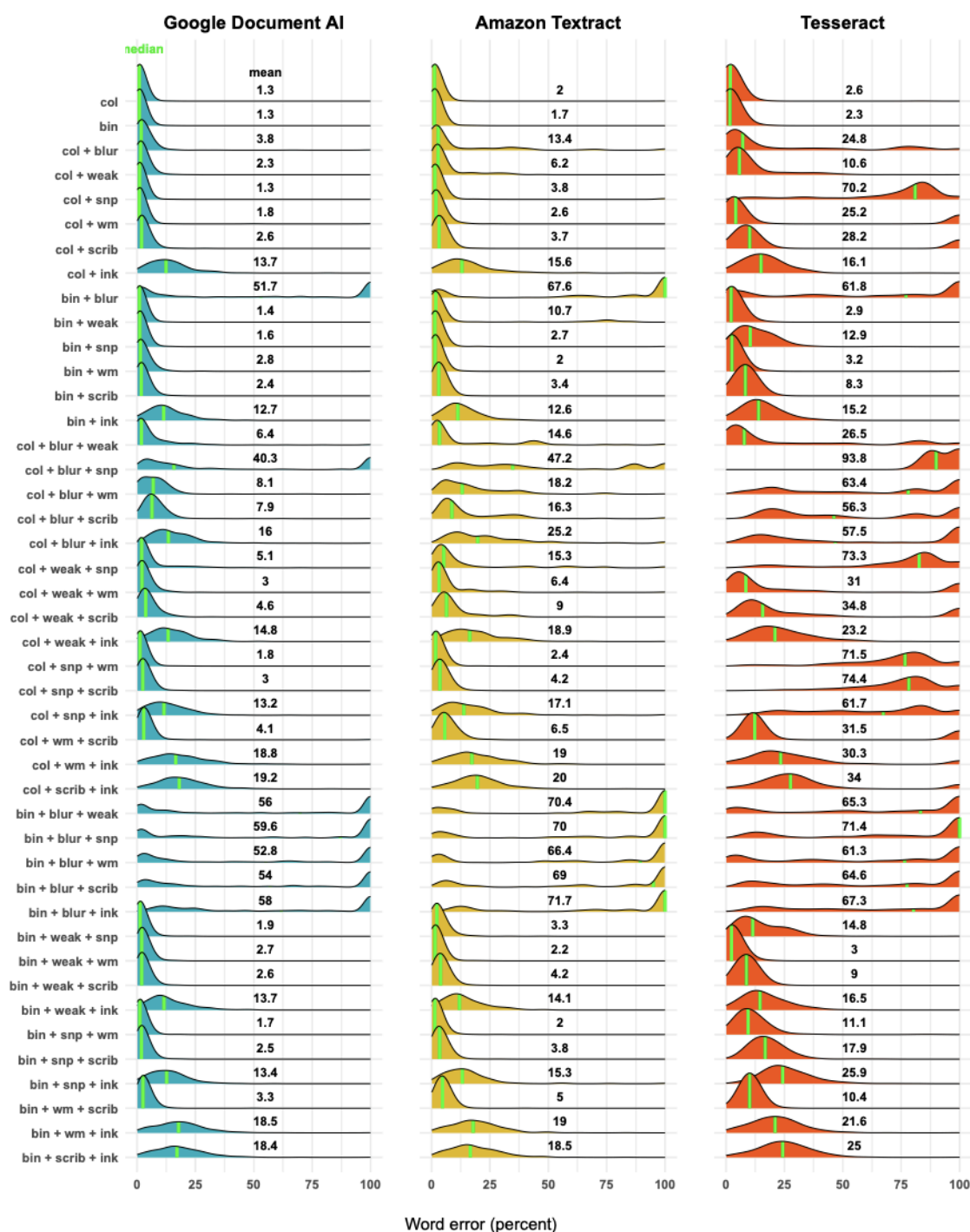


Fig. 5 Word error rates by engine and noise type for English-language documents

5.2 L'integrazione del Large Language Model (LLM)

L'utilizzo esclusivo di un motore OCR presenta un limite intrinseco: l'output è un flusso di testo non strutturato (o debolmente strutturato). Se l'obiettivo fosse estrarre dati standard da documenti identici, si potrebbero utilizzare tecniche posizionali (regex o template matching). Tuttavia, in uno scenario reale con documenti eterogenei, tale approccio risulta rigido e poco scalabile.

Per superare questo vincolo e garantire la generalizzazione del sistema, si è introdotto nella pipeline un Large Language Model (LLM). Il LLM agisce come un layer di interpretazione semantica: riceve in input il testo grezzo ("sporco") fornito dall'OCR e, guidato da un prompt specifico contenente lo schema dei dati desiderati (Detail Set), restituisce un output strutturato (JSON).

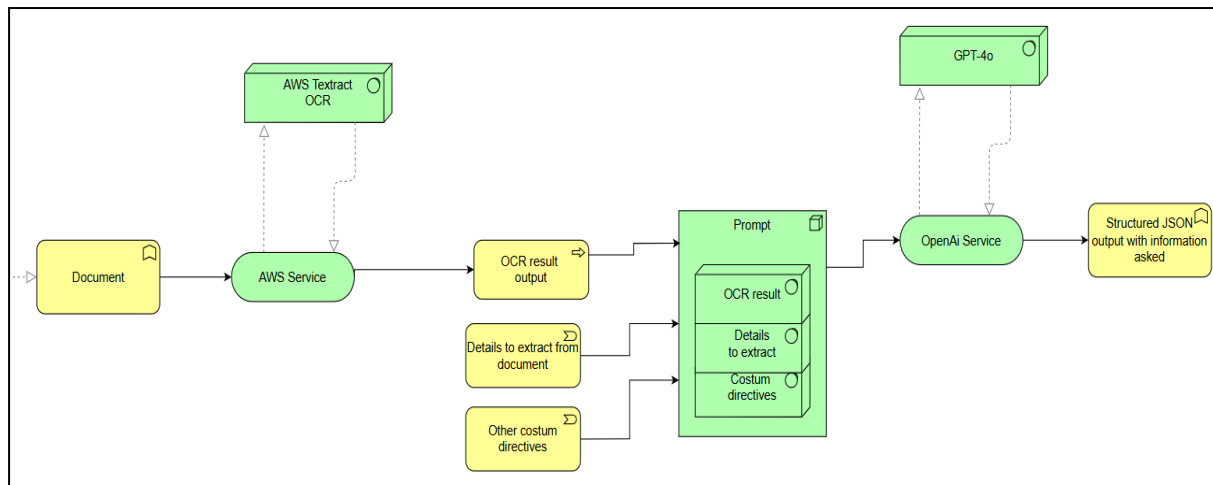
Per questo componente, la scelta è ricaduta sul modello GPT-4o di OpenAI. La decisione è motivata dalla capacità del modello di gestire finestre di contesto ampie e dalla sua elevata capacità di ragionamento zero-shot, necessaria per inferire la natura dei campi (es. distinguere una "data fattura" da una "data scadenza") anche in assenza di training specifico sul layout del documento.

A supporto di tale scelta, un recente studio condotto da Kanerva et al. (2025) ha analizzato le capacità di post-correzione degli errori OCR in contesti zero-shot. I risultati sperimentali dimostrano che, sebbene i modelli open-weight più recenti (come Llama-3.1) abbiano mostrato notevoli miglioramenti, essi rimangono ancora distanti dalle prestazioni dello stato dell'arte commerciale. Nello specifico, GPT-4o ha registrato un miglioramento del Character Error Rate (CER) del 58,1%, distaccando significativamente le soluzioni open-source [15]. Questo dato conferma empiricamente la necessità di adottare un modello di fascia alta per garantire l'affidabilità del dato estratto in contesti aziendali critici. [15]

Model	English		Finnish	
	CER %	WER %	CER %	WER %
Llama-3-8B	7.3	31.4	-68.8	-28.2
Llama-3.1-8B	19.5	37.7	-65.7	-30.1
Llama-3.1-70B	38.7	46.3	-47.0	-8.9
Mixtral-8x7B	-14.9	19.1	-76.5	-40.5
Gemma-2-9B	28.2	38.4	-24.0	-4.1
Gemma-2-27B	35.6	37.8	-19.1	0.0
GPT-4o	58.1	59.1	11.9	33.5

Table 2: Overall CER and WER relative improvement.

6 Descrizione pipeline di estrazione ed elaborazione



Il design della pipeline di elaborazione, illustrato in figura, non è arbitrario ma riflette pattern architetturali emergenti nell'ambito della Document Intelligence moderna. L'approccio adottato, che prevede la serializzazione di un motore OCR ottico seguita da un affinamento semantico via LLM, trova riscontro in soluzioni e librerie di riferimento come "google/langextract" [16], che implementano flussi analoghi per l'estrazione di informazioni strutturate da corpora non strutturati.

Analizzando il flusso dei dati (Data Flow), si identificano i seguenti componenti logici:

Document Input: Rappresenta l'artefatto digitale in ingresso (immagine raster o PDF) che funge da payload per l'intera pipeline.

AWS Service (OCR Gateway): Questa classe agisce come wrapper verso l'infrastruttura cloud di Amazon. Non si limita a inviare la richiesta HTTP contenente il blob dell'immagine, ma incapsula la logica di normalizzazione della risposta, trasformando l'output grezzo di Textract in un formato intermedio standardizzato (che verrà dettagliato nella sezione successiva).

OpenAI Service (Semantic Engine): È il componente responsabile dell'orchestrazione semantica. Questa classe gestisce la costruzione del payload per l'LLM operando una vera e propria Context Injection: aggrega il System Prompt (istruzioni di ruolo), il Detail Schema (la struttura JSON

target) e il testo grezzo proveniente dall'OCR. Il risultato di questa sintesi viene inviato alle API di GPT-4o, che restituisce un oggetto JSON validato e strutturato.

RunExtractionAction

Questa classe (Action) fa comunicare i due servizi per restituire l'output voluto, la logica di trova nel metodo "handle". Esegue inoltre le operazioni di insert nel database di tutte quelle informazioni utili per fini statistici, dati che serviranno per l'analisi dei risultati.

AWS Service

La classe AWS Service contiene un metodo utile per la gestione dell'input e dell'output della API di Textract, il metodo principale è textExtractor che ricevendo come parametro il path del file, genera il Json con la richiesta per Textract tramite un oggetto della classe Aws\Textract\TextractClient

```
$textractClient = new TextractClient([  
    'version' => 'latest',  
    'region' => config('awstextract.region'),  
    'credentials' => [  
        'key' => config('awstextract.key'),  
        'secret' => config('awstextract.secret'),  
    ],  
]);
```

php

poi una volta ritrovato il file tramite path con fopen ed read, viene chiamato il metodo **analyzeDocument** del client di Textract per eseguire la richiesta.

```
$result = $textractClient->analyzeDocument(  
    [  
        'Document' => [  
            'Bytes' => $image,  
        ],  
        'FeatureTypes' => ['TABLES', 'FORMS'],  
    ]  
);
```

php

La sintassi della risposta sarà di questo tipo:

Response AWS Textract

```
{
  "AnalyzeDocumentModelVersion": "string",
  "Blocks": [
    . . .
  ],
  "DocumentMetadata": [
    . . .
  ],
  "HumanLoopActivationOutput": [
    . . .
  ]
}
```

json

```
);
```

Per facilitare il lavoro del Large Language Model viene eseguito un parsing della risposta: vengono eliminati gli elementi superflui come elementi grafici, l'output viene suddiviso in cellMap, rawLines e formFields.

Di seguito è riportato un esempio della struttura **JSON** ottenuta a seguito della fase di parsing del documento. In particolare, il processo produce tre principali componenti:

CellMap array

```
$cellMap = [
  [
    "VAT [%]"      => "10%",
    "Net worth"    => "28 929,32",
    "VAT"          => "2 892,93",
    "Gross worth"  => "31822,25",
    "Net price"    => "3 600,00"
  ],
  [
    "VAT [%]"      => "Total",
    "Net worth"    => "$ 28 929,32",
    "VAT"          => "$ 2 892,93",
    "Gross worth"  => "$ 31 822,25",
    "Net price"    => "177,08"
  ],
  ""
];
```

php

La struttura cellMap rappresenta i dati tabellari estratti dal documento, organizzati in righe e colonne, e consente di preservare la relazione semantica tra le celle individuate.

FormFields array

```
$formFields = [  
    "Tax Id:"          => "",  
    "Invoice no:"      => "58097062",  
    "Client:"          => "Harper-Flynn 753 Acevedo Crossroad Suite 72",  
    "IBAN:"            => "GB79CMCC33561352409593",  
    "VAT [%]"          => "10%",  
    "Knotted Silk Rugs" => "083B",  
    "Net worth"        => "28 929,32",  
    ""  
];
```

php

L'array formFields contiene invece i campi strutturati estratti dal documento, rappresentativi delle informazioni chiave (metadati e valori principali) individuate durante la fase di preprocessing.

RawLines array

```
$rawLines = [  
    "Invoice no: 58097062",  
    "Date of issue:",  
    "04/05/2013",  
    ""  
];
```

php

Infine, rawLines include le singole righe di testo così come estratte dal documento OCR, mantenendo l'ordine originale e fornendo un riferimento testuale non strutturato.

L'insieme di queste strutture JSON viene successivamente incluso nella richiesta inviata all'API di OpenAI, costituendo l'input contestuale per il modello linguistico e permettendo un'estrazione guidata e coerente delle informazioni richieste.

OpenAi Service

Questo service è la classe che gestisce le chiamate a gpt-4o, tramite una libreria chiamata PrismPHP [12].

La chiamate viene eseguita in questo modo:

LLM Structured Output Call

```
Prism::structured()  
    ->using(Provider::OpenAI, 'gpt-4o')  
    ->withSchema($invoiceSchema)  
    ->withPrompt(  
        $prompt->text  
        ."\n\nJson schema: \n\n"  
        . json_encode($invoiceSchema, JSON_PRETTY_PRINT)  
        ."\n\nExtracted text: \n\n"  
        .$invoiceText."\n"  
    )  
    ->asStructured();
```

php

PrismPHP è una libreria utile per strutturare al meglio le chiamate API verso qualsiasi LLM, offre delle dipendenze in grado di sfruttare al meglio i servizi dei vari modelli.

7 Prompt Engineering

7.1 Tecniche di prompt Engineering

Per quanto riguarda le tecniche di prompting, sono state applicate varie tecniche che sono lo stato dell'arte del prompt engineering: zero-shot, few-shot, chains-of-thoughts, una tecnica ibrida e una variante del zero-shot ma con un emotional stimuli.

7.1.1 Zero-shot prompting

Questa tecnica [Radford et al., 2019][24] elimina la necessità di grandi quantità di dati di addestramento, facendo invece affidamento su prompt accuratamente progettati che guidano il modello verso nuovi compiti. In particolare, al modello viene fornita una descrizione del compito all'interno del prompt, ma senza disporre di dati etichettati per l'addestramento su specifiche associazioni input-output. Il modello sfrutta quindi la conoscenza preesistente per generare predizioni in base al prompt fornito per il nuovo compito.

7.1.2 Few-Shot Prompting

Il few-shot prompting fornisce ai modelli alcuni esempi di input-output per indurre la comprensione di un determinato compito, a differenza dello zero-shot prompting, in cui non vengono forniti esempi [Brown et al., 2020][25]. La presenza anche di pochi esempi di alta qualità ha dimostrato di migliorare le prestazioni del modello su compiti complessi rispetto all'assenza di dimostrazioni. Tuttavia, il few-shot prompting richiede token aggiuntivi per includere gli esempi, il che può diventare proibitivo nel caso di input testuali molto lunghi. Inoltre, la selezione e la composizione degli esempi nel prompt possono influenzare in modo significativo il comportamento del modello, e bias come la preferenza per parole più frequenti possono continuare a incidere sui risultati. Sebbene il few-shot prompting potenzi le capacità su compiti complessi, soprattutto nei grandi modelli pre-addestrati come GPT-3, un'attenta progettazione dei prompt è fondamentale per ottenere prestazioni ottimali e mitigare bias indesiderati del modello.

7.1.3 Chain-of-Thought (CoT) Prompting

I Large Language Models (LLM) spesso incontrano difficoltà quando devono affrontare compiti di ragionamento complesso, limitandone il potenziale. Con l'obiettivo di colmare questa lacuna, [Wei et al., 2022][26] hanno introdotto il Chain-of-Thought (CoT) prompting come tecnica per sollecitare gli LLM in modo da facilitare processi di ragionamento coerenti e passo dopo passo. Il contributo principale risiede nella proposta e nell'analisi del CoT prompting, dimostrandone l'efficacia nel generare risposte più strutturate e riflessive rispetto ai prompt tradizionali. Attraverso una serie di esperimenti, gli autori mettono in evidenza le caratteristiche distintive del CoT prompting, sottolineando la sua capacità di guidare gli LLM lungo una catena di ragionamento logica. Ciò porta a risposte che riflettono una comprensione più profonda dei prompt forniti. Ad esempio, il prompt può mostrare il processo di ragionamento e la risposta finale per un problema matematico a più passaggi, imitando il modo in cui gli esseri umani scompongono i problemi in passaggi intermedi logici. Utilizzando prompt CoT con PaLM 540B, gli autori hanno raggiunto prestazioni allo stato dell'arte nei benchmark di ragionamento matematico e di buon senso, ottenendo un'accuratezza del 90,2%.

7.1.4 Forensic + CoT + Structural Awareness (Hybrid)

E' un esempio sofisticato di Chain-of-Thought (CoT) prompting adattato per l'estrazione dati strutturati da output OCR imperfetti. Questa tecnica, supportata da studi come Wei et al. (2022) [26] su CoT per reasoning strutturato e applicazioni OCR in Kojima et al. (2022) su zero-shot CoT, migliora l'accuratezza del 20-50% in task di estrazione da dati rumorosi rispetto a prompt zero-shot, rendendolo ideale per domini forensi ad alta precisione.

7.1.5 Emotion Prompting

Sebbene gli LLM dimostrino capacità notevoli in una vasta gamma di compiti, la loro abilità nel comprendere segnali psicologici ed emotivi rimane incerta. Lo studio di [Li et al., 2023a] ha affrontato questa incertezza introducendo EmotionPrompt, una tecnica volta a migliorare la

comprensione dei segnali emotivi da parte degli LLM. Traendo ispirazione dalla ricerca psicologica sull'impatto del linguaggio sulle prestazioni umane, gli autori aggiungono ai prompt 11 frasi di stimolo emotivo per potenziare l'intelligenza emotiva degli LLM. I risultati sperimentali dimostrano una perfetta integrazione di questi stimoli, con un miglioramento significativo delle prestazioni degli LLM in diversi compiti. In particolare, EmotionPrompt mostra un miglioramento relativo dell'8,00% nell'*instruction induction* e un incremento notevole del 115% nei compiti del benchmark BIG-Bench, evidenziando la sua efficacia nel potenziare le capacità degli LLM nell'elaborazione dei segnali affettivi. Inoltre, una valutazione condotta su 106 partecipanti rivela un miglioramento medio del 10,9% nelle metriche di prestazione, veridicità e responsabilità per i compiti generativi quando si utilizza EmotionPrompt rispetto ai prompt standard.

Application	Prompting Technique	Comparison Scope				
		Prompt Acquisition	Prompt Turn	Language Model(s)	Dataset	Metric(s)
New Tasks Without Training Data	Zero-shot	Manual	Single	GPT-2	Arithmetic, Symbolic	Accuracy, ROUGE Score
	Few-shot	Manual	Single	GPT-3	NaturalQS, WebQS, TriviaQA	Accuracy
Reasoning and Logic	CoT	Manual	Multi	PaLM 540B	GSM8K	Accuracy
	LogiCoT	Manual	Multi	Vicuna-33b, GPT-4	GSM8K, AQuA, SocialQA	Accuracy
	CoS	Manual	Multi	gpt-3.5-turbo, GPT-4	SPARTUN	Accuracy, Precision, Recall
	Auto-CoT	LM Generated	Multi	GPT-3	Arithmetic, Symbolic	Accuracy
	Self-Consistency	Manual	Single	PaLM 540B	Arithmetic, Commonsense	Accuracy
	ToT	Retrieval Based	Multi	GPT-4	Game of 24, Creative Writing	Success Rate
	GoT	Retrieval Based	Multi	T5-large	GSM8K, ScienceQA	ROUGE Score
	S2A	Manual	Single	Llama 2-70B	QA, GSM8K	Accuracy
	ThoT	Hybrid	Multi	gpt-3.5-turbo, Llama 2-70b-chat	PopQA, EntityQ, MTCR	Exact Match (EM) Score
	Chain of Table	Manual	Multi	GPT 3.5, LLaMA 2	TabFact, WikiTQ	BLEU, ROUGE Score
Reduce Hallucination	CoVe	Retrieval Based	Multi	Llama 65B	Wikidata, QUEST, MultiSpanQA	Precision, F1
	ReAct	Retrieval Based	Multi	PaLM-540B, GPT-3	HotpotQA, FEVER	Exact Match (EM), Accuracy
	RAG	Retrieval Based	Single	RAG-Token, RAG-Seq.	MSMARCO, SearchQA	ROUGE, BLEU score
	CoN	LM Generated	Multi	Llama 2, DPR	NO, TriviaQA, WebQ	Exact Match (EM), F1 Score
User Interaction	CoK	LM Generated	Multi	gpt-3.5-turbo-0613	HotpotQA, FEVER, MedMCQA, MMLU Physics and Biology	Exact Match (EM), Accuracy
	Active-Prompt	Manual	Single	code-davinci-002, text-davinci-003	Arithmetic, Commonsense, Symbolic	Disagreement, Entropy Variance, Self-confidence Score
Fine-Tuning and Optimization	APE	LM Generated	Single	text-curie-001, text-davinci-002	BBH, TruthfulQA	Execution accuracy, Log probability, Efficient score estimation
Knowledge-Based Reasoning and Generation	ART	Hybrid	Multi	GPT-3 (175B)	BigBench, MMLU	Accuracy
Improving Consistency and Coherence	CCoT	LM Generated	Multi	gpt-3.5-turbo-0301	Arithmetic, Factual QA	Accuracy
Managing Emotions and Tone	Emotion Prompting	Manual	Single	GPT-4	BIG-Bench, Instruction Induction	Accuracy
Code Generation and Execution	SCoT	Hybrid	Multi	ChatGPT, Codex	HumanEval, MBPP, MBCPP	pass@k
	PoT	Manual	Single	gpt-3.5-turbo	GSM8K, SVAMP, FinQA	Exact Match(EM) Score
	CoC	Manual	Single	text-davinci-003, gpt-3.5-turbo	BIG-Bench Hard	Accuracy
Optimization and Efficiency	Scratchpad Prompting	Manual	Single	GPT-3	MBPP, MBPP-aug	Accuracy
	OPRO	Manual	Single	PaLM 2-L-IT, text-bison	GSM8K, BIG-Bench Hard	Accuracy
Understanding User Intent	RaR	Manual	Single	GPT-4-0613	Knowledge, Symbolic	Accuracy, Fair Score, Language Modeling Score
Metacognition and Self-Reflection	Take a Step Back	Manual	Single	PaLM2-L, GPT-4	MMLU-Physics, MMLU-Chemistry TimeQA, SituatedQA, StrategyQA	Accuracy

Tabella con lista di tecniche di ingegnerizzazione del prompt [16].

8 Evaluation

8.1 Dataset

Il dataset utilizzato per i benchmark sui prompt è costituito da un corpus di 20 documenti.

L'eterogeneità è il criterio fondante di questa raccolta: ogni file è stato selezionato per introdurre specifiche variabili e differenze, permettendo un'analisi completa delle prestazioni.

I documenti sono stati classificati secondo diverse tassonomie (tipologia, formato, qualità, layout, lunghezza e lingua), mantenendo le etichette originali delle classi utilizzate per il testing. Di seguito la distribuzione dettagliata:

- **Document Type:** La varietà tipologica è ampia e include 5 file classificati come Invoice, 3 Receipt, 2 Pay slip, 1 Contract e 2 ID Document. Per testare ambiti non strettamente amministrativi, sono presenti anche 4 documenti di Corporate Intelligence e 3 di tipo Newspaper.
- **File Format:** Per valutare la capacità di gestione dei formati, il dataset comprende 9 Native PDF, 4 Scanned PDF e 7 file di tipo Picture (formati PNG e JPG).
- **Document Quality:** Questa metrica è fondamentale per verificare la robustezza del modello. Sebbene 15 file siano classificati come High-definition, sono stati inclusi casi limite: 3 file Low-definition, 6 Distorted e 7 definiti Noisy (presentano cioè rumore visivo o imperfezioni).
- **Layout:** La struttura dei documenti si divide in due macro-categorie: 8 file sono Prose (testo continuo senza layout fisso) e 12 sono Structured (documenti con formattazione rigida, come tabelle e campi chiave-valore).
- **Word Count:** La lunghezza dei testi varia per testare la "context window" del modello: 7 file rientrano nella classe less than 100 words, 7 sono nel range 100 to 400 words e 6 appartengono alla categoria 400 and more words.
- **Language:** Il dataset è multilingue, composto da 8 file classificati come English e 11 come Italian.

La costruzione del dataset non è casuale, ma mirata al "stress testing": l'obiettivo è mettere in difficoltà il modello per far emergere le caratteristiche del prompt più efficace, specialmente in presenza di classi complesse come Noisy, Distorted o layout non strutturati.

8.2 Benchmark

L'esecuzione del benchmark si basa su una procedura di elaborazione massiva e asincrona, progettata per invocare i metodi dei servizi `AwsTexttractService` e `OpenAiService` sull'intero corpus documentale presente in piattaforma.

Per garantire l'efficienza del processo ed evitare ridondanze, è stato implementato un meccanismo di controllo (check di idempotenza) che esclude dall'analisi i documenti già sottoposti a estrazione in precedenza.

A livello architetturale, la logica della pipeline di estrazione è incapsulata all'interno della classe `RunExtractionAction`. L'esecuzione effettiva viene orchestrata dal job `BulkRunJob`, il quale si occupa del dispatch asincrono della action per ogni documento idoneo.


```
class RunExtractionAction
```

php

```
{
    public function handle(Document $document, Prompt $prompt): void
    {
        // Creazione del record della run
        $run = Run::create([
            'document_id' => $document->id,
            'prompt_id'   => $prompt->id,
        ]);

        // Estrazione testo tramite AWS Textract
        $extracted_text = app(AwsTextractService::class)
            ->textExtractor($document->getFirstMediaPath('document'));

        // Elaborazione strutturata tramite OpenAI
        $structured_result = app(OpenAIService::class)
            ->readInvoice(
                $extracted_text,
                $document,
                $prompt
            );

        // Salvataggio dei campi estratti nel database
        foreach ($structured_result as $key => $result) {
            $documentDetail = DocumentDetail::whereName($key)->first();

            if ($documentDetail) {
                $document->extractedFields()->create([
                    'document_detail_id' => $documentDetail->id,
                    'run_id'              => $run->id,
                    'value'               => $result,
                ]);
            }
        }
    }
}
```

Al termine del processo di estrazione, i valori recuperati risultano disponibili in piattaforma e associati all'entità Document Detail di ciascun file. La fase successiva consiste nella validazione dei risultati, attuata mediante il confronto puntuale tra i valori estratti e i valori attesi definiti nei Ground Truth.

Relativamente alla gestione dei dati di riferimento, si specifica che il popolamento dei Ground Truth avviene in una fase distinta e successiva all'inserimento del documento in piattaforma, venendo definito specificamente per ogni singola istanza documentale.

8.3 Definizione della Metrica di Valutazione (Score)

La metrica adottata in questo studio, denominata Score, quantifica il grado di correttezza semantica dell'estrazione informativa rispetto al ground truth.

Per ogni campo estratto, si assegna un valore compreso nel seguente intervallo:

Score va da 0.0 a 1.0

- 1.0: Corrispondenza perfetta.
- 0.0: Nessuna corrispondenza semantica.

8.3.1 Calcolo dello score

Per calcolare lo score delle estrazioni sono stati applicati 3 criteri di valutazione in base alla tipologia del dato estratto (stringa, data, numero).

Per quanto riguarda le stringhe viene utilizzato il metodo

```
similar_text(  
    string $string1,  
    string $string2,  
    float &$percent = null  
): int
```

Questo calcola la somiglianza tra due stringhe come descritto in Programming Classics:

Implementing the World's Best Algorithms di Oliver (ISBN 0-131-00413-1)[19]. Questa implementazione non utilizza uno stack come nel pseudocodice di Oliver, ma chiamate ricorsive che possono o meno velocizzare l'intero processo. Nota inoltre che la complessità di questo algoritmo è $O(N^3)$ dove N è la lunghezza della stringa più lunga. [18]

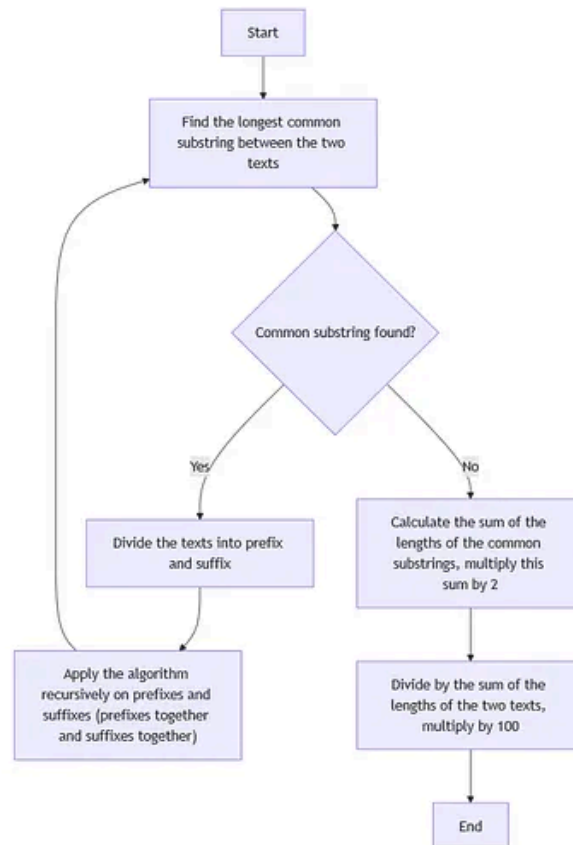


Immagine presa dal blog medium.com [19]

Il motivo di questa scelta è che l'algoritmo non tiene conto dell'ordine delle parole, lo scopo del benchmark non è testare l'OCR ma il Large Language Model nel riuscire a comprendere le direttive ed il significato semantico dell'input, quindi non interessa se estrae "Rossi Mario" invece di "Mario Rossi", ma interessa che abbia compreso che il cliente sia Mario Rossi.

Confronti tra algoritmi più famosi:

Algoritmo	Ordine Parole	Complessità	Adatto per LLM?
similar_text	Ignora	$O(N^3)$	Sì
Levenshtein	Considera	$O(N^2)$	No
Jaccard	Ignora	$O(N)$	Sì
Ukkonen	Considera	$O(N)$	No

`Similar_text()` è ottimale per valutare la comprensione semantica dell' LLM, ignorando l'ordine lessicale ma catturando il contenuto informativo.[20]

Per i campi che richiedono un'interpretazione univoca, come date e valori numerici, l'utilizzo di metriche di similarità testuale (es. `similar_text`) può generare falsi positivi fuorvianti. Di conseguenza, sono state adottate logiche di validazione "strict" (rigide) basate sul parsing dei valori.

Gestione dei campi Data Per le entità di tipo Date, il confronto viene effettuato previa normalizzazione degli oggetti temporali mediante la libreria Carbon [23]. Questa scelta è dettata dalla necessità di evitare ambiguità sintattiche. Un algoritmo di similarità tra stringhe, infatti, giudicherebbe erroneamente simili due date semanticamente distinte.

- Esempio di falso positivo (String Similarity): Confrontando "23/02/2025" (Valore A) e "3/02/2025" (Valore B), una metrica testuale restituirebbe uno score elevato (es. 0.9/1.0) a causa della sovrapposizione dei caratteri.

- Approccio adottato (Strict Semantic Match): Poiché non è determinabile a priori se la discrepanza sia dovuta a un errore di allucinazione del LLM o a un difetto OCR (es. mancata lettura di una cifra), il sistema impone una corrispondenza esatta. Nel caso sopra citato, lo score assegnato sarà tassativamente 0 / 1.

Per i valori monetari e quantitativi, il confronto diretto tra stringhe rischia di generare falsi negativi dovuti alla formattazione. Il sistema applica quindi una pipeline di sanitizzazione e normalizzazione prima del confronto:

1. Sanitizzazione: Rimozione di simboli di valuta (€, \$, £) e caratteri non numerici.
2. Normalizzazione: Standardizzazione dei separatori delle migliaia e dei decimali (es. conversione da 1.234,56 a 1234.56).

Al termine di questo processo, il confronto avviene sottraendo allo score massimo (1.0) la differenza tra i due valori decimali (estratto e atteso) in valore assoluto, se risulta negativo lo score è zero.

Implementazione:

```
Compare number function php  
protected function compareNumbers(?string $extractedValue, string $expectedValue)  
{  
    $cleanedExtracted = preg_replace('/[^\\d.]/', '', $extractedValue);  
    $cleanedExpected = preg_replace('/[^\\d.]/', '', $expectedValue);  
    $floatEx = (float) str_replace(',', '.', $cleanedExtracted);  
    $floatExp = (float) str_replace(',', '.', $cleanedExpected);  
    $diff = abs($floatEx - $floatExp);  
    $score = 1.0 - $diff;  
    return $score < 0 ? 0.0 : $score;  
}
```

Esempio:

```
extracted=2049.17,  
expected=2049.167,  
diff=0.0030000000001564,  
score=0.99699999999984
```

8.3.2 Considerazioni sugli algoritmi scelti

L'algoritmo di Oliver, pur essendo efficace per misurare la somiglianza tra stringhe, è altamente sensibile alla presenza di caratteri extra. Pertanto, un modello che fornisce la risposta corretta ma con una formattazione leggermente diversa (es. spazi aggiuntivi o prefissi) viene penalizzato maggiormente rispetto a un output grezzo ma pulito.

9 Risultati

Type / Prompt	Zero-Shot	Few-Shot	Chain-of-Thought	Forensic + CoT + Structural Awareness	Zero-Shot Emotional
date	0.7504	0.7504	0.7887	0.7504	0.7277
number	0.9302	0.8372	0.9048	0.7500	0.8718
string	0.8578	0.8411	0.8456	0.8493	0.8170
Tag / Prompt	Zero-Shot AVG: 0.8701	Few-Shot AVG: 0.8326	Chain-of-Thought AVG: 0.8587	Forensic + CoT + Structural Awareness AVG: 0.8115	Zero-Shot Emotional AVG: 0.8252
100 to 400 words	0.8626	0.8056	0.8356	0.7757	0.8118
400 and more words	0.8349	0.7982	0.8487	0.7689	0.7636
Contract	0.9388	0.9441	0.9441	0.9441	1.0000
Corporate Intelligence	0.7582	0.7509	0.7555	0.6581	0.6825
Digital	0.8731	0.8433	0.8580	0.8254	0.8257
Distorted	0.8901	0.8497	0.8929	0.8562	0.8689
English	0.9052	0.8895	0.9155	0.8335	0.8642
High-definition	0.8848	0.8679	0.8932	0.8240	0.8603
ID Document	0.8949	0.8747	0.8547	0.8547	0.8436
Invoice	0.8680	0.8098	0.8282	0.7986	0.8090
Italian	0.8434	0.7892	0.8195	0.7928	0.7924
Low-definition	0.8320	0.7791	0.8315	0.8515	0.7887
Native PDF	0.8689	0.8368	0.8475	0.7748	0.8146
Newspaper	0.8940	0.8372	0.9159	0.8359	0.8821
Noisy	0.9065	0.8652	0.9005	0.8558	0.8777
Pay slip	0.9558	0.9155	0.9558	0.9155	0.9184
Picture	0.8557	0.8194	0.8317	0.8506	0.8149
Prose	0.8414	0.8135	0.8621	0.7732	0.7995
Receipt	0.8847	0.8672	0.8609	0.9407	0.8565
Scanned PDF	0.9256	0.8620	0.9608	0.8599	0.9173
Structured	0.8876	0.8442	0.8569	0.8421	0.8387
less then 100 words	0.9138	0.9014	0.8960	0.9212	0.8912
Average accuracy	0.8701	0.8326	0.8587	0.8115	0.8252

9.1 Presentazione dei risultati

Una volta eseguito lo scoring delle estrazioni verrà creata una tabella con un riassunto delle estrazioni, nella tab Benchmark Stats dell'applicazione, dove si possono consultare tutte le statistiche e i dati grezzi dell'estrazione. Il valore presente nella tabella è la media degli score ottenuta per ogni categoria.

Dall'analisi dei dati riportati in tabella emerge la seguente distribuzione dei punteggi medi di accuratezza (Score) associati alle diverse strategie di prompting. L'approccio Zero-shot registra il valore più elevato (0.8701), seguito dal Chain-of-Thought (0.8587). Prestazioni inferiori si osservano per il Few-shot (0.8326) e per lo Zero-shot Emotional (0.8252), mentre l'approccio Hybrid risulta il meno performante, con un punteggio medio pari a 0.8115.

Differenziando l'analisi in base al metodo di scoring adottato, emerge uno scenario differente.

Per i campi di tipo stringa, valutati tramite l'algoritmo Oliver basato sulla funzione `similar_text`, l'approccio Zero-shot ottiene il valore di accuratezza più elevato (0.8578), seguito dall'Hybrid (0.8493) e dal Chain-of-Thought (0.8456). I risultati del Few-shot (0.8411) e dello Zero-shot Emotional (0.8170) risultano invece inferiori.

Per quanto riguarda i campi numerici, valutati mediante l'algoritmo `compareNumbers`, l'approccio Zero-shot si conferma il più performante, raggiungendo un'accuratezza pari a 0.9302. Seguono il Chain-of-Thought (0.9048) e lo Zero-shot Emotional (0.8718), mentre il Few-shot (0.8371) e, in particolare, l'Hybrid (0.7500) mostrano prestazioni sensibilmente più basse.

Come avviene nella maggior parte dei test sperimentali condotti sui large language model, le differenze nei risultati ottenuti adottando approcci differenti risultano generalmente contenute. In particolare, analizzando i punteggi medi, si osserva una variazione dell'accuratezza di circa il 6% tra l'approccio Zero-shot e quello Hybrid. Nonostante ciò, l'analisi dei dati evidenzia trend interessanti e mette in luce differenze rilevanti tra i diversi prompt utilizzati.

9.2 Analisi dei risultati

9.2.1 Analisi sullo score medio dei prompt

Un'analisi preliminare dei risultati suggerisce una conclusione controintuitiva: l'aumento della complessità del prompt non correla positivamente con l'accuratezza dell'output del Large Language Model. Al contrario, le strategie più elaborate sembrano introdurre un degrado nelle performance.

Per indagare le cause delle prestazioni inferiori osservate nella tecnica ibrida, è stata inizialmente formulata l'ipotesi della “attention dilution”. Tale ipotesi trae ispirazione dai risultati e dalle considerazioni emerse dallo studio di Liu, Nelson F., et al., *Lost in the Middle: How Language Models Use Long Contexts*, secondo cui fornire ai language model contesti di input più estesi costituisce un compromesso. Se da un lato una maggiore quantità di informazioni può facilitare l'esecuzione del task a valle, dall'altro incrementa il volume di contenuto su cui il modello è chiamato a ragionare, aumentando il rischio di una riduzione dell'accuratezza. [23]

Questa tesi può trovare fondamento anche se analizziamo gli score del Few-shot, che nonostante abbia istruzioni più dettagliate, regole ed esempi da seguire, controintuitivamente performa peggio del Zero-shot, vediamo i due prompt a confronto.

Definizione Zero-shot (255 caratteri, 49 token):

```
### TASK INSTRUCTIONS
```

```
Your task is to extract structured data from the OCR text provided below.
```

```
Analyze the text and fill in the requested fields according to the provided JSON schema.
```

```
### RESPONSE
```

```
Return ONLY the valid JSON object matching the schema.
```

Definizione Few-shot (3966 caratteri, 1280 token):

ROLE

Act as an expert data corrector.

CORRECTION RULES & ADVICE

1. OCR engines often confuse similar characters (e.g., "5" with "S", "1" with "I", "." with ","). Your task is to extract the data and correct these errors based on context.

2. OCR engines extract raw data, so the year could be truncated like 12/08/25. In those cases, the year is "2025" and not "1925", since we are in the 2020s.

3. If a field represents a number like "Total amount", "Age", or "Net worth" and the extracted text is, for example, "1234o,5", the "o" should be corrected to "0", number can't have letters.

4. The extracted text is provided in 3 parts: cellMap, formFields, rawLines. CellMap contains the cells identified with values inside the cells. formFields contains all forms of the document identified by the OCR engine, with their value (this is the most important part for analyzing the content). RawLines contains the lines of the document without formatting.

EXAMPLES (FEW-SHOT LEARNING)

Example 1:

- Json schema:

```
{
  "name": "document",
  "description": "Document Schema",
  "properties": [
    { "name": "Receipt number", "description": "", "nullable": false },
    { "name": "Date", "description": "", "nullable": false },
    { "name": "Product count", "description": "", "nullable": false },
    { "name": "Total amount", "description": "", "nullable": false }
  ],
  "requiredFields": [],
```

```
    "allowAdditionalProperties": false,  
    "nullable": false  
}
```

- Extracted text:

```
$cellMap [  
  { "Receipt No": "n.12534", "": "" },  
  { "Date": "05/12/25", "": "" },  
  { "Pizza Qt.": "1", "": "" },  
  { "Torta Qt.": "2", "": "" },  
  { "Costo totale: " : "760,5$", "": "" }  
]
```

```
$formFields [  
  { "field": "Receipt number", "value": "12534" },  
  { "field": "Date", "value": "05/12/25" },  
  { "field": "Pizza Qt.", "value": "1" },  
  { "field": "Torta Qt.", "value": "2" }  
  { "field": "Costo tale:", "value": "760,5" }  
]
```

```
$rawLines [  
  "Receipt No: 12534",  
  "Date: 05/12/25",  
  "Pizza Qt.: 1",  
  "Torta Qt.: 2",  
  "Costo totale: 760,5$"  
]
```

Structured response:

```
{
  "Receipt number": "12534",
  "Date": "05/12/2025",          # this is an example of how to
correct a date
  "Product count": 3,           # quantity of Pizza (1) + quantity
of Torta (2) = 3, example of a calculated field
  "Total amount": 760.5         # replace "0" with "0" because the
total amount is a number"
}
```

****Example 2:****

- Json schema:

```
{
  "name": "document",
  "description": "Document Schema",
  "properties": [
    { "name": "Invoice number", "description": "", "nullable": false },
    { "name": "Date", "description": "", "nullable": false },
    { "name": "Item count", "description": "", "nullable": false },
    { "name": "Total amount", "description": "", "nullable": false }
  ],
  "requiredFields": [],
  "allowAdditionalProperties": false,
  "nullable": false
}
```

- Extracted text:

```
$cellMap [  
  { "Invoice No": "INV12078", "": "" },  
  { "Date": "15/10/25", "": "" },  
  { "Laptop Qt.": "2", "": "" },  
  { "Mouse Qt.": "3", "": "" },  
  { "Total amount": "3045,90€", "": "" }  
]
```

```
$formFields [  
  { "field": "Invoice number", "value": "INV12078" },  
  { "field": "Date", "value": "15/10/25" },  
  { "field": "Laptop Qt.", "value": "2" },  
  { "field": "Mouse Qt.", "value": "3" },  
  { "field": "Total amount", "value": "3045,90" }  
]
```

```
$rawLines [  
  "Invoice No: INV12078",  
  "Date: 15/10/25",  
  "Laptop Qt.: 2",  
  "Mouse Qt.: 3",  
  "Total amount: 3045,90€"  
]
```

Structured response:

```
{  
  "Invoice number": "INV12078",      # OCR correction: "0" -> "0"
```

```

    "Date": "15/10/2025",           # year corrected from 25 -> 2025
    "Item count": 5,                # Laptop (2) + Mouse (3) = 5
    "Total amount": 3045.90         # OCR corrections: "0" -> "0" and
    comma to decimal
  }

```

RESPONSE

Based on the rules and examples above, extract the data from the RAW OCR INPUT and return ONLY the valid JSON object.

Ai caratteri e ai token utilizzati è inoltre necessario aggiungere il testo estratto formattato e il file JSON contenente i dettagli da estrarre, entrambi inclusi nel prompt finale. Il conteggio complessivo risulta variabile a causa dell'eterogeneità del dataset e dipende principalmente dalla lunghezza del testo analizzato. In base a controlli effettuati su un campione rappresentativo, tale valore varia indicativamente tra i 3000 e i 4000 caratteri e tra i 1000 e i 1500 token.

Come emerge dalle definizioni dei prompt, si osserva una differenza significativa in termini di numero di token e di complessità tra le due configurazioni. Nonostante ciò, l'accuratezza media risulta a vantaggio dell'approccio Few-shot. Tale tendenza è confermata anche dall'analisi degli score per singolo algoritmo di comparazione, all'interno della quale il Few-shot mostra complessivamente prestazioni superiori.

Alla luce dei risultati ottenuti, è possibile attribuire ulteriore validità allo studio citato in precedenza e affermare che, per questo specifico task di estrazione di dettagli da testo proveniente da un sistema OCR, l'adozione di un approccio Zero-shot risulta complessivamente più conveniente rispetto al Few-shot. Tale scelta è motivata non solo dal consumo significativamente inferiore di token richiesto dallo Zero-shot, ma anche dal fatto che l'approccio Few-shot, a causa dell'elevato numero di token coinvolti, può favorire fenomeni di "attention dilution", con un impatto negativo sull'accuracy. Tale conclusione vale anche per altri approcci token-consuming come il CoT e l'Hybrid (Forensic + CoT + Structural Awareness).

	Zero-shot	Few-shot	CoT	Hybrid	Zero-shot E.
Caratteri	225	3966	957	1974	385
Token	49	1280	221	484	75

Nella tabella sono raffigurati i conteggi dei caratteri e dei token consumati dal modello Gpt-4o dei vari prompt.

9.2.2 Sensibilità della Metrica

Dai test effettuati, emerge un dato controintuitivo: il Prompt Zero-shot presenta uno score di similarità superiore rispetto a tecniche più evolute come il Chain-of-Thought. Questa evidenza può essere attribuita alla natura della funzione `similar-text()` utilizzata per la valutazione. Mentre i prompt complessi inducono il modello a una maggiore verbosità o all'aggiunta di descrittori testuali, lo Zero-shot produce un output estremamente sintetico. In un contesto di estrazione dati dove la precisione del singolo carattere è fondamentale, la semplicità del prompt ha ridotto drasticamente il rumore statistico nell'output finale.

Prompt	Document	Detail name	Extracted value	Ground truth value	Score
Zero-Shot	Global Finance Wire	Key Person Role	Marcus V. Thorne, CEO	Marcus V. Thorne (CEO)	0.93023255813953
Few-Shot	Global Finance Wire	Key Person Role	Marcus V. Thorne, CEO of OmniCorp Logistics	Marcus V. Thorne (CEO)	0.61538461538462

L'immagine è stata estratta dalla tabella dei dati grezzi presente nella pagina "Benchmark Stats" del framework utilizzato per l'esecuzione dei benchmark. Essa evidenzia come l'approccio Few-shot aggiunga descrittori al valore estratto, determinando di conseguenza una riduzione dello score ottenuto (30%).

9.3 Analisi specifiche sui risultati di ogni prompt

9.3.1 Analisi di Zero-shot

Come già evidenziato, la tecnica Zero-shot risulta quella con l'accuratezza media più elevata su tutte le metriche considerate. I suoi punti di forza risiedono nella capacità di effettuare estrazioni dirette, senza necessità di calcoli intermedi: in particolare, si dimostra infallibile sui valori numerici, registrando un'accuratezza media del 93%, superiore di circa il 3% rispetto al Chain-of-Thought (CoT) e del 18% rispetto all'approccio ibrido.

Analizzando l'accuratezza in funzione della tipologia di documento, si osserva che il Zero-shot viene superato dal CoT nei documenti più lunghi di 400 parole, nelle pagine di giornale, nei PDF scannerizzati e nei documenti in lingua inglese; mentre l'approccio ibrido mostra migliori performance negli scontrini e nei documenti con meno di 100 parole.

Sulla base di questi dati, e considerando i documenti in cui lo Zero-shot ha ottenuto un'accuratezza inferiore rispetto ad altri prompt, documenti tipicamente più lunghi, meno strutturati e con maggiore necessità di interpretazione, si può concludere che, per input più complessi e caotici, un approccio più rigoroso (ma non troppo verboso) risulti generalmente più efficace.

9.3.2 Analisi di Few-shot

L'approccio Few-shot, che teoricamente dovrebbe rappresentare la strategia più performante (cfr. Capitolo 7.1.2 Few-Shot Prompting), risulta tuttavia, in termini di accuratezza media, tra i meno efficaci. Tale comportamento può essere attribuito a diversi fattori. In primo luogo, la qualità degli esempi forniti: essendo il dataset eterogeneo, risulta complesso selezionare un numero limitato di esempi rappresentativi e di alta qualità, poiché ciascun tipo di documento produce un output differente. Ciò può generare bias nei confronti di documenti non presenti nel cluster degli esempi selezionati. In secondo luogo, un ulteriore fattore potrebbe essere il fenomeno di attention dilution, dovuto alla verbosità e all'elevato numero di token del prompt generato. Tuttavia, l'approccio Few-shot mostra performance rilevanti in specifici task, come l'estrazione di dettagli da

contratti, testi in inglese, buste paga, PDF scannerizzati e documenti con meno di 100 parole, registrando uno score medio superiore a 0.85.

Altro punto a favore di questo approccio è la qualità dell'output fornito, infatti in un contesto più aziendale o integrando questo sistema in software più complessi potrebbe servire un output con un formato specifico, ed è qui che questo approccio mostra le sue qualità.

Prompt	Document	Detail name	Extracted value	Ground truth value	Score
Zero-Shot	Identity card	Birth Date	14.02.1999	14/02/1999	0.8
Few-Shot	Identity card	Birth Date	14/02/1999	14/02/1999	1

Infatti grazie agli esempi e alle regole notiamo che restituisce la data nel formato indicato nel prompt, o comunque in un formato standard.

Nonostante ciò, nel complesso, per questo specifico task, esso non risulta essere la strategia più adatta.

9.3.3 Analisi del Chain-of-Thought

In linea generale, questo approccio può essere considerato il più accurato sulla maggior parte dei task, superando il 90% di accuratezza in sei diverse categorie. Confrontandolo con il primo classificato, risulta più performante sui documenti che richiedono maggiore interpretazione, come fogli di giornale, PDF scannerizzati e contratti, come evidenziato in precedenza.

Ciò suggerisce che esso rappresenti un compromesso ideale tra complessità e verbosità del prompt, consumando circa un migliaio di token in meno rispetto al Few-shot, pur mantenendo un'accuratezza notevole anche sui task più complessi. Lo score peggiore è stato registrato nella categoria Corporate Intelligence, in cui però tutti i prompt hanno incontrato difficoltà. Pertanto, se si considera la categoria in cui questo approccio ha ottenuto prestazioni inferiori rispetto agli altri, risulta essere la categoria Picture, ovvero i documenti in formato PNG o JPG, differenza poco significativa visto che si aggira sul 2% rispetto al Zero-shot e all'approccio ibrido.

9.3.4 Analisi del Forensic + CoT + Structural Awareness (Hybrid)

In media, questo approccio risulta essere il meno performante. Tale scarsa prestazione può essere attribuita alla complessità del prompt, che combina diverse tecniche e può indurre il modello a generare allucinazioni. Nonostante ciò, considerando la metrica dell'algoritmo Oliver, esso ha ottenuto uno score generale di 0.8493, superando il Chain-of-Thought (CoT) (0.8456) e differenziandosi dal Zero-shot di soli 0.8 punti percentuali.

Il vero punto critico di questo approccio riguarda la gestione dei numeri: nella maggior parte dei casi, infatti, vengono restituiti valori con decimali errati, scale non corrette o formati inappropriati.

Prompt	Document	Detail name	Extracted value	Ground truth value	Score
Forensic + CoT + Structural Awareness	Invoice	Total amount	31.82225	31822,25	0
Forensic + CoT + Structural Awareness	Invoice	Net worth	28.92932	28929,32	0
Forensic + CoT + Structural Awareness	CONFIDENTIAL INTERNAL MEMORANDUM	Financial amount	68000000	58000000	0

Tale comportamento può essere attribuito al fatto che, a causa dell'elevato numero di direttive presenti nel prompt, il modello mostra difficoltà nel determinare il corretto processo di “ragionamento” da applicare a ciascun dato specifico, aggiungendo complessità laddove non necessaria.

9.3.5 Analisi dello Zero-Shot with Emotional stimuli

Nel test in oggetto, l'approccio Zero-shot con stimolo emozionale non migliora l'accuratezza delle risposte rispetto all'approccio classico. In tutti i campi analizzati, infatti, questo metodo mostra performance inferiori, ad eccezione della categoria Contratti, dove si registra un leggero miglioramento. Una possibile spiegazione a questo comportamento, analizzando la tabella dei dati grezzi, potrebbe derivare dal fatto che con lo stimolo emozionale il modello è spinto maggiormente a trovare una risposta, anche dove questa non è necessaria.

Prompt	Document	Detail name	Extracted value	Ground truth value	Score
Zero-Shot Emotional	Invoice 3	Document number	TD01 fattura	Unknown	0
Zero-Shot Emotional	Pay slip 2	Payment date	31/08/2021	August 2021	0.38095238095238
Zero-Shot Emotional	Receipt 3	Payment type	Cash or Card based on rounding adjustment note	Unknown	0

Nella tabella, la presenza del valore “Unknown” nella colonna “Ground Truth Value” indica che il relativo dato non è presente all'interno del documento.

In conclusione lo stimolo emozionale in questo task non è utile, visto l'alto tasso di precisione che il modello dovrebbe avere. D'altra parte, si osserva che lo stimolo emozionale ha comunque un effetto sul comportamento del modello, in quanto lo induce a “fare qualcosa”. Nel nostro caso, questa direzione è opposta all'obiettivo del task, ma il modello viene comunque guidato in una direzione specifica. Ciò suggerisce che, con versioni più personalizzate e attentamente studiate dello stimolo emozionale, le performance del modello potrebbero potenzialmente migliorare.

10 Conclusione

In conclusione, la ricerca dimostra che per l'ottimizzazione di documenti fiscali tramite Large Language Models, l'efficacia del Prompt Engineering non segue un andamento lineare rispetto alla complessità delle istruzioni fornite. Al contrario, si è osservato che l'accuratezza è inversamente proporzionale alla verbosità richiesta dal prompt.

Nello specifico, per task di estrazione puntuale (Information Extraction), la configurazione Zero-shot si è dimostrata la più robusta, garantendo un'accuratezza media del 87%. Questo risultato, sebbene possa apparire controintuitivo rispetto ai trend recenti che vedono nel Few-shot e nel Chain-of-Thought (CoT) lo standard aureo, trova una giustificazione tecnica precisa nella natura dei dati trattati:

1. Riduzione della deriva dell'output (Output Drift): I documenti fiscali richiedono una corrispondenza esatta di stringhe brevi (date, importi, Partite IVA). Mentre i prompt più articolati spingono il modello a generare ragionamenti intermedi o testi di cortesia (es. "Il valore estratto è..."), lo Zero-shot costringe l'LLM a un output atomico, minimizzando il rumore statistico rilevato dall'algoritmo di Oliver.
2. Mitigazione delle allucinazioni: La maggiore libertà semantica concessa ai modelli nelle configurazioni Few-shot ha talvolta portato a "allucinazioni di formato", dove il modello tentava di uniformare i dati estratti agli esempi forniti nel prompt piuttosto che attenersi fedelmente al testo grezzo restituito dall'OCR.
3. Efficienza e Scalabilità: Dal punto di vista ingegneristico, la superiorità dello Zero-shot implica una riduzione significativa dei token di input (meno costi e minore latenza) e una semplificazione dell'architettura software, non rendendo necessaria la gestione di complessi dataset di esempi per il context window.

Queste evidenze suggeriscono che, nel dominio specifico della correzione OCR per documenti strutturati, la strategia ottimale non risiede nell'aumento della complessità delle istruzioni, ma nella precisione sintattica del comando iniziale.

Analizzando i sottodomini del task in oggetto, è possibile individuare le tecniche di prompt engineering maggiormente consigliate, secondo i dati emersi dalla presente ricerca:

- Estrazione di stringhe / Comprensione del testo: Zero-shot, con un'accuratezza media dell'85%.
- Estrazione e calcolo di valori numerici: Zero-shot, con un'accuratezza media del 93%.
- Estrazione di date: Chain-of-Thought, con un'accuratezza media del 79%.

Questi risultati evidenziano come l'efficacia delle diverse strategie di prompting sia fortemente dipendente dal tipo di informazione da estrarre, suggerendo che la scelta della tecnica debba essere adattata al sottodominio specifico.

Nel complesso, le prestazioni osservate delle diverse tecniche di prompt engineering sono elevate ma non dipendono esclusivamente dall'efficacia del prompt stesso. Questo perché il prompt rappresenta soltanto una componente della pipeline di ottimizzazione del testo estratto dall'OCR, una pipeline solida che garantisce un input di qualità superiore rispetto al testo raw generato dall'OCR, grazie anche alla strutturazione rigorosa dei JSON forniti al prompt.

In conclusione, l'obiettivo di questa tesi è stato dimostrare come, all'interno dello stesso contesto altamente strutturato e di qualità, la formulazione delle direttive all'interno di un prompt possa influenzare in maniera significativa la qualità dell'output ottenuto, migliorandola o compromettendola a seconda della correttezza delle indicazioni fornite.

Bibliografia e Sitografia

- [1] R. Mittal and A. Garg, "Text extraction using OCR: A Systematic Review," *2020 Second International Conference on Inventive Research in Computing Applications (ICIRCA)*, Coimbatore, India, 2020, pp. 357-362, doi: 10.1109/ICIRCA48905.2020.9183326.
- [2] PHP Documentation Group, "PHP: What is PHP? - Manual," [Online]. Disponibile: <https://www.php.net/manual/it/intro-what-is.php>. [Accesso: Gennaio 2025].
- [3] PHP Documentation Group, "PHP: What can PHP do? - Manual," [Online]. Disponibile: <https://www.php.net/manual/it/intro-what-can-do.php>. [Accesso: Gennaio 2025].
- [4] Taylor Otwell, "Laravel Documentation (v12.x)," [Online]. Disponibile: <https://laravel.com/docs/12.x>.
- [5] Filament PHP, "Filament Documentation," [Online]. Disponibile: <https://filamentphp.com/docs>.
- [6] Caleb Porzio, "Livewire Quickstart Guide (v3.x)," [Online]. Disponibile: <https://livewire.laravel.com/docs/3.x/quickstart>.
- [7] L. Aversano, G. Canfora, L. Cerulo, C. Grosso, and M. Di Penta, "An empirical study on the evolution of design patterns," *Proceedings of the 6th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering*, 2007, pp. 385-394. doi: 10.1145/1287624.1287680.
- [8] ACM, "Proceedings of the European Conference on Pattern Languages of Programs 2020," Association for Computing Machinery, New York, NY, USA, 2020. doi: 10.1145/3424771.3424773.
- [9] PHP.eearth, "Object-Oriented Programming (OOP) Introduction," [Online]. Disponibile: <https://docs.php.eearth/php/ref/oop/intro/>.
- [10] T. Hegghammer, "OCR with Tesseract, Amazon Textract, and Google Document AI: a benchmarking experiment," *Journal of Computational Social Science*, vol. 5, pp. 861–882, 2022. doi: 10.1007/s42001-021-00149-1.
- [11] Spatie, "Laravel MediaLibrary - Documentation," [Online]. Disponibile: <https://github.com/spatie/laravel-medialibrary>.
- [12] PrismPHP, "Prism: Large Language Models for Laravel," [Online]. Disponibile: <https://github.com/prism-php/prism>.
- [13] Taylor Otwell, "Laravel Queues and Jobs," [Online]. Disponibile: <https://laravel.com/docs/queues>.
- [14] J. Kanerva, C. Ledins, S. Käpyaho, and F. Ginter, "OCR Error Post-Correction with LLMs in Historical Documents: No Free Lunches," *Proceedings of the Third Workshop on Resources and Representations for Under-Resourced Languages and Domains (RESOURCEFUL-2025)*, Tallinn, Estonia, 2025, pp. 38–47.
- [15] Google Open Source, "Langextract: A library for language detection," [Online]. Disponibile: <https://github.com/google/langextract>.
- [16] P. Sahoo et al., "A systematic survey of prompt engineering in large language models: Techniques and applications," *arXiv preprint arXiv:2402.07927*, 2024.
- [17] PHP Documentation Group, "PHP: similar_text - Manual," [Online]. Disponibile: <https://www.php.net/manual/function.similar-text>.
- [18] L. Dorn, "Understanding the PHP similar_text algorithm," *Medium*, 2021. [Online]. Disponibile: <https://medium.com/@leo.dorn/understanding-the-php-similar-text-algorithm-904808c2bc20>.
- [19] I. Oliver, *Programming Classics: Implementing the World's Best Algorithms*. Prentice Hall, 1994. ISBN 0-131-00413-1.
- [20] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning*. Springer, 2013. Cap. 2.2.3 (The Bias-Variance Trade-off).

- [21] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006. Cap. 1.5 (Decision Theory).
- [22] Carbon PHP Group, "Carbon: A simple PHP API extension for DateTime," [Online]. Disponibile: <https://carbon.nesbot.com/>.
- [23] LIU, Nelson F., et al. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 2024, 12: 157-173. [Online]. Disponibile: <https://arxiv.org/abs/2307.03172>
- [24] Xiao, Y. and Jin, Z.Z. (2021) Summary of Research Methods on Pre-Training Models of Natural Language Processing. *Open Access Library Journal*, **8**, 1-7. Disponibile: [10.4236/oalib.1107602](https://oalib.org/oalib/1107602).
- [25] BROWN, Tom, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 2020, 33: 1877-1901. Disponibile: <https://doi.org/10.48550/arXiv.2005.14165>
- [26] WEI, Jason, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 2022, 35: 24824-24837. Disponibile: <https://arxiv.org/abs/2201.11903>